

Optimizing the performance of Database as a Service (DaaS) model- A distributed approach

Thara R J

Dept. of Computer Science
College of Engineering, Trivandrum
Trivandrum, India
tharaneelima@gmail.com

Shine S

Dept. of Computer Science
College of Engineering, Trivandrum
Trivandrum, India
shine.s2008@gmail.com

Sanu C

Dept. of Computer Science
College of Engineering, Trivandrum
Trivandrum, India
sanuchandramohan@gmail.com

Abstract—Database as a Service (DaaS) is one among the latest service models offered by cloud computing and is adapted by most of the organizations. Database fragmentation, replication and allocation are the key issues to be tackled by most of the DaaS framework. Out of these issues the problem of replication is discussed throughout this paper. Optimizing the performance of DaaS service model by means of strategic database replication is the motivation behind this paper. We assume that the database fragmentation is already done for a given DaaS platform, and we take database fragments as the unit of replication in our replication procedure. As part of the survey, we studied the existing database replication schemes in detail and identified their major performance issues. After review we introduced a new strategy for database replication in DaaS framework. Our approach solves some of the current issues involved and targets to mixed goals out of replication. The replication strategy proceed in three phases; selecting the most popular database fragment, its placement at optimal server and finally the maintenance of replicas forever. Simulation experiments are conducted to evaluate the performance of proposed algorithm. We observed that our algorithm runs in polynomial time and yields suboptimal solution.

Keywords—Database as a Service (DaaS); database fragment replication; popular fragment selection; database fragment replica placement; cloud computing.

I. INTRODUCTION

Distributed cloud computing provides an open platform to deal with large scale data intensive as well as computation intensive applications. Cloud computing adopts the divide and conquer paradigm which makes it a suitable architecture for distributed parallel computing, file sharing etc. With the divide and conquer approach, complex computations are handled efficiently by dividing them into small manageable pieces and distributing across individual nodes. Nodes can now conquer these sub problems with little effort, and finally the solutions are combined back to yield solution to original problem.

Cloud computing offers a number of service models such as Platform as a Service (PaaS), Software as a Service (SaaS) etc. One recent service model of cloud computing is Database as a Service (DaaS), where we can store and manipulate huge databases with little effort. Database fragmentation, replication and allocation are the key processing essential to keep DaaS

perform better. Out of these the problem of replication is studied and tackled by this paper.

The success of the cloud DaaS paradigm relies on how they satisfy their applications' demands in terms of reliability and query access efficiency. Reliable system with improved access efficiency can be achieved by means of proper replication. We choose database fragments as the unit of replication. Fragment replication creates multiple copies of critical fragments, and stores them in appropriate locations. Fragmenting the huge database into smaller ones and keeping the fragments closer to user applications can reduce the service time with improved local access probabilities. Further availability guarantees and fault tolerance can be achieved by means of proper replication. To get the maximum performance out of database replication; some strategic placement of fragment replicas based on the applications' requirements is needed. An intelligent placement of fragment replicas will definitely outperform some random placement of replicas. Our study focuses on proposing such an intelligent database replication strategy for distributed cloud data base environments.

The recent applications from DaaS model assume the highly scalable distributed data store management strategy, referred to as NOSQL (Not Only SQL). The increasing usage of non-relational databases among web applications is commonly subsumed under the term NOSQL. Horizontal scaling, availability, partitioning and replication of data among servers are the major attractions of such systems. NOSQL achieves higher performance guarantees and scalability by giving up the ACID constraints of traditional database management schemes; and follows a more relaxed BASE constraint, i.e. Basically Available, Soft state and Eventual consistency. In most of their applications, the NOSQL systems give up consistency; and try to retain availability and partition tolerance. Data replication has a major role on achieving availability and fault tolerance, and thereby improving the overall performance of NOSQL based DaaS paradigm. We studied the concept of replication in the context of DaaS paradigm offering NOSQL services.

In addition to data replication in distributed data stores, the concept of replication is studied in various distributed computing domains including grid, peer to peer, distributed files etc. The method as well as the major features of replication are studied, and analysed the problems with existing approaches. The method of replication will vary from domain

to domain, and in worst case from applications to applications as the ultimate goal of replication varies. When one tries to achieve a specific goal, it may force him to give up others. Our proposed replication strategy combines some of the recent goals together. The method is proposed exclusively to cloud computing applications, especially for the service model DaaS; but with slight modification we can apply our basic concept to other distributed computing environments like peer to peer, grid etc.

We can think of any replication procedure to proceed in three key phases: replica selection, replica placement and replica management phases. Replica selection procedure selects the deserved candidate for replication; now replica placement procedure identifies the optimal locations for placing the selected replicas. The last phase is replica management, i.e. how efficiently we manage the replicated copies once the replication process is over. Replica consistency and transparency issues are tackled here, and it is a life time process. Consistency ensures correctness of update propagation; where transparency indicates how efficiently the system masks its complexities from users.

The rest of the paper is organized as follows: section 2 we survey the existing work on data replication, Section 3 models the proposed system; we start our discussion with the system model assumed followed by brief explanation of each and every module in our proposed method. Section 4 gives the simulation results and result analysis. Finally section 5 concludes the overall work performed.

II. LITERATURE SURVEY

The replica placement problem has been studied extensively in many disparate fields of distributed computing such as grid computing[6], peer to peer computing[8], cloud computing[20] etc. Availability and efficient access are the critical requirements in many data intensive applications. Delayed access due to availability problems and non responsiveness may cause undesired results. To efficiently address these challenges, data replication is essential. A good replication algorithm can be used to anticipate or analyze the user requests for data and to place the subsets of data and replicas at strategic locations. It is difficult to make manual decisions about where the data needs to be placed. An automated data replication system is needed that can take into account the data access pattern by multiple users and applications. In addition to achieving access efficiency and availability data replication can also be used to improve data locality and increase the robustness and scalability of many applications.

M Shorfuzzaman et al. [7] developed an algorithm for the replica placement in hierarchical data grids. The goal of their algorithm is to increase the access performance through the dynamic creation of replicas of popular files. Here, popularity is determined by examining the recent history of access rates by analyzing the file access histories. The algorithm will be executed at regular intervals and it analyses the file access history, identifies the popular files and determines the new replica locations. This strategy should ensure that the popular files are retained when new replicas arrive. To achieve this,

popularity based form of Least Recently Used (LRU) is used with a constraint added to ensure that the replicas created in the current replication interval is not replicated.

M Shorfuzzaman et al. [3] proposed a modified version of [7], where the creation of replica is based on the dynamic adjustment of threshold value, as in [7], the threshold value is fixed. In [7], the efficiency of replica placement depends heavily on the selection of threshold value which determines the popularity of the data object. So [3] determines the threshold value dynamically after considering factors like access request arrival rate from the clients and the available storage capacity of the replica servers. Through this dynamic adjustment of the threshold, the tradeoff between the access latency and the storage capacity of the servers are balanced.

Zeinab and Amir [8] considered the replica placement in graph based data grid network. The goal is to balance the load in the replica servers with the objective of minimizing access latency and communication cost. It estimates the round trip time and available band width between the host pairs. The algorithm is executed in three phases. Phase 1 traverse the graph using BFS. Let the BFS tree be T . The adjacent of each node of T is kept. Run DFS to traverse the tree. Maintain the list L of nodes in the same order as they are visited. Phase 2 performs replica Selection and placement based on L . Finally phase 3 carry out LRU based replacement if needed.

Jian Zhou et al. [9] proposed a static, distributed replica placement algorithm in P2P networks. The algorithm considers the replica placement in real world P2P networks as a clustered K center problem which is NP complete. It is an efficient approximation algorithm which can work with several orders of magnitude faster than optimal solutions with minimum access latency. The clustered k center problem is similar to classic k center problem in graph theory. The k center problem tries to find a set of k centers in an arbitrary graph such that the maximum shortest distance of all nodes to the nearest center is minimized.

Hai Jiang et al.[11] discussed an efficient replica placement algorithm for the Hybrid CDN-P2P(Content distribution networks – P2P) architecture, or HCDN, which combines the complementary advantages of CDN and P2P networks. Such a network can reduce the deployment cost faced by the pure CDN networks and can improve the quality of service in file sharing and video streaming applications compared to pure P2P networks. The CDN-P2P hybrid network is thus an efficient architecture for large-scale content distribution. A heuristic replica placement algorithm that takes into account the effects of P2P distribution at P2P level as well as the deployed surrogate servers at CDN level has been discussed. Compared to replica placement algorithms for pure CDN networks, the approach taken here can reduce the placement cost (i.e. the deployment cost), as the peer contribution is taken into account in this approach.

An alternative and better strategy for placing replicas strategically in HCDN with assumed ring topology is proposed in [14]. Here each surrogate server makes the replica placement in terms of the content replicas on k closer surrogate servers, which enhances the system scalability

compared to the centralized replica placement heuristics. Their approach tries to obtain maximum gain out of replications.

III. PROPOSED SYATEM

Throughout our work we propose a strategic approach for database fragment replication in real cloud network. Optimizing the performance of Data base as a Service (DaaS) model by means of efficient data replication motivated our work. We start our discussion with the basic assumption, followed by modeling and detailed analysis of proposed strategy.

A. Problem Formulation

The fragment replication problem solved in this paper assumes the cloud network having set of servers with dedicated memory to store the replicated copies of database fragment. Figure 1 shows the system model we assumed. Cloud manager is the centralized authority responsible for managing the cloud. Replica management is added as an additional module to cloud manager API, called cloud replica manager. The module once we run will keep track of the creation, placement and management of replicas.

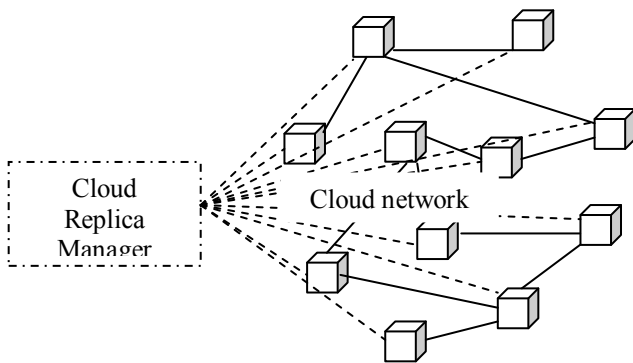


Figure1: System Model

To propose our strategy we assume tree topology for the entire cloud network with root node acting as the origin server and leaves dedicate their memory o replicate fragments. Whenever some query arrives at a server, it tries to resolve the query by accessing the fragment kept locally. If it fails to satisfy the query with the available fragments kept locally, it forwards the request to its nearby servers and tries to resolve the query from its neighboring servers. In worst case when the query fails to resolve by all the nearby servers, the request will be satisfied by the origin server, which is assumed to have the entire data stores.

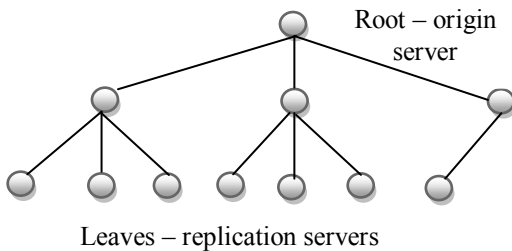


Figure 2 –topological view of cloud

B. Database as a Service (DaaS) Paradigm

Cloud computing offers technology enabled services to people and organizations, which enable them to access resources online, from anywhere at any time without worrying about technical/physical management issues of the original resources. The attractive feature of this new distributed computing paradigm is their service models including Software as a Service (SaaS), Platform as a Service (PaaS) and Infrastructure as a Service (IaaS). DaaS service model is emerging as a new promising offering from cloud computing; where one can store and manipulate huge databases efficiently and cost effectively. DaaS follows distributed relational modeling for managing huge databases. The key issues regarding the performance of DaaS service model are database fragmentation, replication and allocation. Fragmentation means dividing the relation into a number of sub relations, called fragments. Fragments are distributed across the network and user requests are redirected towards the fragments, which results reduced access time and bottleneck overhead compared to traditional schemes. Popular fragmentation schemes are horizontal fragmentation and vertical fragmentation. Once the fragmentation process completes, the next question is how to replicate and allocate the fragments across the cloud. Replication means creating multiple copies of database fragments, where allocation means the placement of fragments and their replicas in optimal locations.

C. NOSQL Data stores

Distributed data store management is adopted by most of the day to day applications, especially social network services like face book, twitter etc. The movement is generally termed as Not Only SQL abbreviated as NOSQL. The approach is entirely different in concept with relational database approaches; which is preferred by applications requiring high degree of accuracy like banking applications. The ACID (Atomicity, Consistency, Integrity and Durability) constraints are not there in NOSQL movement; rather they assumes BASE (Basically Available, Soft state and Eventual consistency) model to support their applications. Availability is the major requirement for applications that prefer NOSQL data stores. In order to achieve the desired level of availability such applications simply sacrifice the ACID constraints and happy with the BASE model provided by NOSQL. The idea is that by giving up ACID constraints, one can achieve much higher performance and scalability.

In the context of NOSQL one can cite Eric Brewer's CAP theorem which states that a system can have only two out of three of the following properties: consistency, availability, and partition-tolerance. Depending on the application one can chose the combination of any two features. The NOSQL systems generally give up consistency; and try to retain availability and partition tolerance. However the systems differ in how much they give up. For example, usually we talk about two variants of consistency models: strict and eventual consistency models. But most of the systems lie in between the two extremes, meaning that updates are eventually propagated to all nodes, but many of them provide

mechanisms for some degree of consistency, such as multi-version concurrency control (MVCC).

D. Fragment replication – an overview

Availability and reliability guarantees are the major goals requested by distributed data stores; especially cloud DaaS paradigm. Replication plays a major role in keeping the system fault tolerant and thereby reliable. Overall availability of the system also gets increased as a result of replicated copies. In this paper we propose a strategic approach for database fragment replication. The fragment replication strategy proceeds in three key phases: selection of candidate fragments, creation and placement of replicas at optimal servers. An overview of each of the phases is illustrated below.

E. Fragment replica selection

The first phase is the selection of unit of replication i.e. database fragments. Prior to replication procedure we take the assumption that database already get fragmented appropriately. Now from the set of available fragments replica selection procedure selects the optimal fragment, i.e. the one with maximum gain out of replication. The selection criteria can be any performance measures; we chose popularity of fragments as the selection criterion. Popularity of fragments is indication of its access rate, i.e. the frequency by which each fragment gets accessed. Once we collect the popularity of fragments, we sort fragments nonincreasingly according to their popularity values and select fragments for replication in the sorted order.

F. Fragment replica placement

Once we select and replicate the optimal fragment, the next phase is placement of fragment replica. Fragment replica placement is the process of selecting optimal network site to place the replica. The term optimal site means the selected site should yield the desired level of performance expected from the cloud. Our procedure assumes limited capacities for the servers. So in worst case, it may happen that the selected optimal server reaches its capacity limit. In such a situation we should talk about replacement strategies. We adopt a simple strategy of replacing the less popular object whenever eviction is required. There may be situations that the optimal server reaches its capacity limit and the selected fragment which is going to replicate is of low popularity level compared to the popularity of already placed fragments in the optimal server. In such a situation we simply ignore eviction and precede our strategy with the next server which found to be optimal among the remaining servers. Our process terminates when all the servers reach their limit and filled with most popular objects.

G. Fragment replica management

Finally our third phase is replica management or maintenance phase; which consumes 75 % time of the overall strategy. Replication consistency and transparency are the critical activities of maintenance phase. Replica consistency means how the system makes the replicated copies up-to-date, i.e. mechanisms for propagating updates efficiently across

replicated copies of the objects should be there and the schemes should impose the required level of consistency across the system. Depending on the application we can either choose strict or eventual consistency models. In strict consistency model, the replica manager prohibits all the read / write accesses involved in all the replicas of critical fragments whose changes are propagating among its replicas. Strict consistency rules are adopted by applications like banking transactions; where accuracy of data accessed is crucial than all other features. To achieve the desired level of accuracy, the system may sacrifice the delay involved and makes its applications ready to face considerable latencies involved for propagating the updates.

Distributed computing applications, especially distributed databases prefer eventual consistency models for their applications. The tight constraints while propagating updates among replicas are relaxed by such applications; rather they provide a more liberal atmosphere for propagating updates. They allow their applications to have read / write access to critical data, once the updates are being propagating among its replicas; i.e. both update propagation and read / write access proceed in parallel. NOSQL movement chooses eventual consistency model as their default consistency model.

The maintenance of replication transparency is another issue handled by replica maintenance phase. Users should be given a transparent view of the system, i.e. the user request should be satisfied without complicating him with the details of replication; he should feel that there is no replication within the system and the request were satisfied from the original fragment. Hiding the complexity of replication from users is a difficult task and it's better to assign the task to middleware module in all distributed applications; which is responsible for masking the heterogeneity among servers and to provide a transparent view of system. We consider the maintenance phase of replication as future work to our study.

Reliable performance of the system with respect to availability and fault tolerance is crucial in most of the distributed applications including distributed database management. Redundancy plays a major role in achieving the desired level of reliability for the system. In addition to optimized reliability, the proposed solution should guarantee reduced access latency to critical data in worst case.

In the context of distributed databases, once the database gets fragmented, the next step is to have a proper replication strategy to yield optimum performance. Our proposed strategy for fragmentation, replication and allocation is given in the following session.

IV. SIMULATION RESULTS

Simulation was carried out using OMNeTpp simulator. OMNeTpp is an object-oriented modular discrete event network simulation framework. It can be used in various problem domains such as modelling of wired and wireless communication networks. Models are assembled from reusable components termed modules. Modules can be connected with each other via gates and combined to form compound modules. We have developed a reconfigurable emulator of cloud network on a 2D plane with the help of OMNet++ to test

our strategy. In our experiment initially 22 nodes are arranged as a k ary complete tree (assumed topology) within a 400x600 rectangular area, we take $k = 3$ during our experiment. All nodes are assumed to have same configuration. We repeat our experiment many times consecutively to test the correctness of our strategy.

V. CONCLUSIONS

An efficient database replication strategy for optimizing the performance of cloud Database as a Service (DaaS) model is proposed in this paper. Existing approaches for replication is studied under distributed computing environments like grid computing, peer to peer computing and cloud computing. The significance of database fragment replication is studied under the cloud computing service model DaaS and NOSQL data stores. Improving the performance of distributed database service model provided by cloud computing motivated our work. The proposed strategy for database fragment replication and Replica placement is simulated using OMNeT++ simulator. We extend our study on similar distributed applications as our future work.

REFERENCES

- [1] Myunghoon Jeon ; Kwang-Ho Lim, "G. Dynamic Data Replication Scheme in the Cloud Computing Environment", Network Cloud Computing and Applications (NCCA), 2012
- [2] Rashid F, Miri A, "A secure data deduplication framework for cloud environments" Privacy, Security and Trust (PST), 2012
- [3] M. Shorfuzzaman, P. Graham, and R. Eskicioglu. "Adaptive placement of replicas in hierarchical data grids". In Proceedings of the High Performance Computing Symposium (HPCS'10), Jun 2010
- [4] Wenhao Li , Yun Yang , "A Cost-Effective Mechanism for Cloud Data Reliability Management Based on Proactive Replica Checking", Cluster, Cloud and Grid Computing (CCGrid), 2012
- [5] Aiqiang Gao, Luhong Diao, "Lazy update propagation for data replication in cloud computing", 5th International Conference on Pervasive Computing and Applications (ICPCA), 2010.
- [6] K Sashi, Dr. Antony Selvadoss Thanamani, "Dynamic Replica Management for Data Grid", IACSIT International Journal of Engineering and Technology, August 2010.
- [7] M. Shorfuzzaman, P. Graham, and R. Eskicioglu. "Popularity-driven dynamic replica placement in hierarchical data grids". In Proceedings of the 9th International Conference on Parallel and Distributed Computing, Applications and Technologies (PDCAT'08), Workshop on HPDataGrid, Dec 2008.
- [8] Zeinab Fadaie, Amir Masoud Rahmani, Islamic Azad University, Tehran, Iran, "A new Replica Placement Algorithm in Data Grid", IJCSI International Journal of Computer Science March 2012
- [9] Jian Zhou, Xin Zhang, Laxmi N. Bhuyan, Bin Liu. "Clustered K-Center: Effective Replica Placement in Peer-to-Peer Systems." In proceeding of the Global Communications Conference, 2007
- [10] Mahesh Maurya, Ketan Shah, "A Review on File replication Algorithms", Techno-Path Journal of Sci. Engg. & Tech. Mgt. Vol 3(2), July 2011
- [11] Hai Jiang; Zhan Wang; Wong, A.K.; Jun Li; Zhongcheng Li. "A Replica Placement Algorithm for Hybrid CDN-P2P Architecture". Parallel and Distributed Systems (ICPADS), 2009 15th International Conference, Page(s): 758 – 763
- [12] Yu-Chih Tung; Lin, K.C.; Cheng-Fu Chou. "Bandwidth-Aware Replica Placement for Peer-to-Peer Storage Systems". Global Telecommunications Conference (GLOBECOM 2011), 2011 IEEE, Page(s): 1 - 5
- [13] Rzdca, Krzysztof; Datta, Anwitaman; Buchegger, Sonja. "Replica Placement in P2P Storage: Complexity and Game Theoretic Analyses". Distributed Computing Systems (ICDCS), 2010 IEEE 30th International Conference, 2010 , Page(s): 599 – 609.
- [14] Zhan Wang; Hai Jiang; Yi Sun; Jun Li; Jing Liu; Dutkiewicz, Eryk. "A k-coordinated decentralized replica placement algorithm for the ring-based CDN-P2P architecture," Computers and Communications (ISCC), 2010 IEEE Symposium – Page(s): 811 - 816. 2010
- [15] Yu-Chih Tung; Lin, K.C.; Cheng-Fu Chou. "Bandwidth-Aware Replica Placement for Peer-to-Peer Storage Systems", Global Telecommunications Conference , IEEE, 2011
- [16] Ed Sharrukh Zaman,,Daniel Grosu, Wayne State University, Detroit. "A Distributed Algorithm for the Replica Placement Problem" IEEE Transactions on Parallel and Distributed Systems – (vol. 22 no. 9) pp. 1455-1468 . September 2011.
- [17] Sharrukh Zaman,,Daniel Grosu, Wayne State University, Detroit. "A Distributed Algorithm for the Replica Placement Problem" IEEE Transactions on Parallel and Distributed Systems – (vol. 22 no. 9) pp. 1455-1468 . September 2011.
- [18] Lin, W.K. ; Ye, C. ; Chiu, D.M, "Decentralized Replication Algorithms for Improving File Availability in P2P Networks," Quality of Service, 2007 Fifteenth IEEE International Workshop, 2007.
- [19] Khafa, F. ; Kolici, V et al., "Data Replication in P2P Collaborative Systems," P2P, Parallel, Grid, Cloud and Internet Computing (3PGCIC),Seventh International Conference, 2012
- [20] Dawei Sun, Guiran Chang , "Building a High Serviceability Model by Checkpointing and Replication Strategy in Cloud Computing Environments", Distributed Computing Systems Workshops (ICDCSW), 2012